

Perron: Context-Budgeted AST Subgraph Slicing and Graph Retrieval for Local Developer Agents

Sultan Haikal

yvliet@users.noreply.github.com

Google - The Gemma 4 Developer Agent Paper Track

October 1, 2026

Abstract

Autonomous software engineering agents powered by open-weights Large Language Models (LLMs) face severe hardware and graph-topological bottlenecks when deployed in resource-constrained local environments: repository call graphs exhibit heavy-tailed power-law degree distributions. Standard multi-hop graph expansions (such as 2-hop Breadth-First Search) frequently encounter ubiquitous utility functions, base classes, and loggers, expanding the retrieval set to over 2,500 symbols (>50,000 tokens) and overflowing the model’s 4,096-token context window. Furthermore, unconstrained 128K KV caches consume 8–12 GB RAM, triggering severe swap thrashing and Out-Of-Memory (OOM) failures on standard 16GB consumer laptops.

To resolve this dilemma, I introduce **Perron**, an open-source AST subgraph slicing and graph retrieval architecture designed around a **Dual-Tier deployment model**: (1) an Edge Deployment Tier targeting **Gemma 4 E4B (4B)** via quantized GGUF execution, requiring only 2.4 GB of weights and 0.4 GB KV cache (operating within 10.4 GB total resident RAM including OS and background tools), reserving 5.6–9.2 GB headroom on 16GB consumer laptops for test subprocesses and compilers; (2) a Workstation Tier scaling to **Gemma 4 26B A4B** (Sparse MoE with 3.8B active parameters) and **Gemma 4 31B** on 24GB GPUs; (3) Zero-Copy Memory-Mapped Transition Matrix Ingestion, compiling call and caller dependencies into static row-stochastic CSR matrices serialized as raw binary arrays, achieving ingestion in under 1.0 ms (<170 KB RAM overhead on 6,000-symbol repositories); (4) Query-Directed Specificity Ratio Diffusion, eliminating hub-node context explosion without mutating static matrix entries by contrasting issue-directed Personalized PageRank (π_{query}) against a precomputed repository stationary baseline (π_{global}) via smoothed Pointwise Mutual Information; and (5) Versioned Frontier Packing paired with an AST-grounded editor that enforces indentation-tolerant block rebasing and pre-commit syntax validation. Benchmarked across 50 real SWE-bench Lite issue instances on authentic repository call graphs (Requests: 284 symbols, 770 edges; SymPy: 6,033 symbols, 17,938 edges), Perron achieves a 95.6% Hub Suppression Index (a statistically significant improvement over Standard PPR, paired $t(49) = 5.02$, $p < .001$, Cohen’s $d = 0.71$, Cliff’s $\delta = 0.38$), 42.0% File Recall@4k (matching lexical baselines), and 4.0% Function Recall@4k (4.0% vs. 0.0% for Standard PPR), while running with sub-5 ms CPU diffusion (<1.0 ms on Requests, 5.07 ms on SymPy). Full end-to-end defect repair runs offline with zero syntax errors, establishing a reproducible, publication-grade foundation for local developer agents.

1 Introduction

Evaluating autonomous software development agents on benchmarks like SWE-bench [5] requires models to localize defects across large codebases, generate targeted patches, and pass execution-based test suites. While cloud agents rely on remote APIs, local open-weights deployment faces physical hardware memory limits and prompt context boundaries ($K \leq 4,096$ tokens).

1.1 The Hardware Reality: 16GB Consumer Laptops vs. 24GB Workstations

A central premise of developer agent democratization is enabling individual engineers to run agents locally on standard developer laptops (16GB RAM) without cloud API costs or private IP leakage. While models like Gemma 4 natively support a 128K token context window, maintaining an unconstrained 128K KV cache requires 8–12 GB in FP16 (or 4.8 GB under INT4 quantization). On a 16GB laptop, this triggers severe operating system swap thrashing or Out-Of-Memory termination alongside prohibitive autoregressive time-to-first-token (TTFT) latency (>30s per turn).

To ensure continuous, real-time developer workflows, Perron bounds the active per-turn working context to $K_{\text{effective}} = 3,480$ tokens ($K \leq 4,096$), restricting the INT4 KV cache to ~0.4 GB (scaling to ~0.8 GB under multi-turn history) and sustaining <1.5s generation latency per turn. Under this bounded working memory envelope, hardware viability is bifurcated:

- **Gemma 4 31B (INT4 Quantized):** 15.5 GB model weights + 0.8 GB bounded KV cache + 4.0 GB OS and system memory = **20.3 GB RAM minimum**. This requires single 24GB workstations (RTX 3090/4090 or Apple Silicon M-series Max).
- **Gemma 4 E4B (4B PLE, INT4 GGUF):** 2.4 GB model weights + 0.4 GB bounded KV cache + 0.8 GB GGML scratch compute buffers + 6.5 GB OS and IDE memory = **10.4 GB resident RAM footprint on typical developer laptops** (leaving **5.6 GB true free RAM headroom** on 16GB machines for concurrent AST parsing and targeted test subprocesses; or up to 9.2 GB headroom under minimal base OS environments).

Perron reconciles this physical reality via a **Dual-Tier deployment architecture**:

- **Tier 1 (Edge Local):** Defaults to Gemma 4 E4B (and E2B for edge nodes) executed locally via lightweight GGUF runtimes (`perron/backends/gguf_backend.py`) with zero GPU requirements and native Chain-of-Thought <|think|> token extraction.
- **Tier 2 (Workstation Scaled):** Targets Gemma 4 26B A4B (activating 3.8B parameters per token across 128 experts) and Gemma 4 31B on 24GB GPUs (RTX 3090/4090 or Kaggle P100/T4 instances) for enterprise repositories requiring deep multi-file refactoring.

1.2 The Topological Bottleneck: Scale-Free Dependency Graphs

Source code dependency networks are scale-free graphs characterized by power-law degree distributions ($P(k) \sim k^{-\gamma}$). Ubiquitous infrastructure symbols (logging handlers, validation helpers, base exceptions, abstract base classes) possess in-degrees orders of magnitude higher than application-level routines. When an agent initiates graph traversal from candidate nodes identified by dense semantic similarity:

1. The search frontier encounters a hub node within 1 to 2 hops.
2. Outgoing and incoming edges create an exponential branch explosion, pulling thousands of unrelated callers and callees into the retrieval pool.
3. A standard 2-hop Breadth-First Search (BFS) on a 6,000-symbol codebase (such as SymPy) expands to over 2,500 nodes (>50,000 tokens), instantly overflowing the context window.
4. Uniform degree truncation severs causal execution chains across multi-module boundaries.

Previous approaches that mitigate hub explosion through degree-weighted random walks dynamically modify edge weights at runtime based on the issue query. However, dynamic matrix reweighting invalidates

memory-mapped storage, requiring full matrix reallocation in RAM for each query. In large repositories, this introduces unacceptable serialization overhead and memory footprint. Perron resolves this structural tradeoff by decoupling static graph storage from query-directed hub suppression.

2 Related Work

Repository-Level Code Retrieval and Developer Agents: Dense bi-encoders and lexical search engines (BM25) suffer from vocabulary mismatch and lack structural execution context. RepoCoder [14] demonstrated on RepoEval (1,600 line completion tasks across 8 repositories) that augmenting retrieval queries with generated code across sliding windows (S_w, S_s) improves zero-shot completion by $> 10\%$. However, its iterative generate-then-retrieve loop requires multiple sequential LLM inference passes ($C_{\text{ret}}^1 \rightarrow \hat{Y}^1 \rightarrow C_{\text{ret}}^2 \rightarrow \hat{Y}^2$), creating prohibitive latency and token inflation under tight prompt limits ($K \leq 4,096$). AutoCodeRover [15] showed that navigating AST representations (classes, methods, caller/callee chains) resolves 22.0% of SWE-bench Lite issues, but delegates navigation to iterative LLM agent search queries. In a 300-issue ablation on SWE-bench, SWE-agent [13] proved that agents interacting with a bare Linux shell resolve only 1.8% of issues due to lack of bounded viewing windows and unvalidated edits, whereas custom Agent-Computer Interface (ACI) commands with immediate syntax feedback boost resolve rates by +10.7 percentage points to 12.5%. Agentless [12] proved that multi-turn autonomous agents taking 30 to 40 turns suffer from error propagation and lack effective self-reflection, showing that a simple two-phase pipeline of hierarchical localization followed by test-filtered patch generation achieves 27.33% on SWE-bench Lite at \$0.34 per issue. Perron synthesizes these findings by executing fine-grained AST symbol-level graph retrieval and context packing in a single deterministic pass.

Graph-RAG and Random Walk Diffusion: Random walk formulations have a rich foundation in graph representation learning, initiated by DeepWalk [9] and Personalized PageRank [8]. Unsupervised graph mining toolkits, such as Karate Club [10], and recent knowledge graph foundation models [1] emphasize scalable sparse matrix operations over complex relational topologies. In information retrieval, HippoRAG [2] demonstrated on multi-hop QA benchmarks (MuSiQue, 2WikiMultiHopQA) that single-step Personalized PageRank over knowledge graphs outperforms iterative retrieval (IRCoT) by up to 20% while running 10-30 $\times$ cheaper and 6-13 $\times$ faster, tuning PPR damping to $\beta = 0.5$ and synonymy to $\tau = 0.8$. HippoRAG formulated node specificity as inverse passage frequency $s_i = |P_i|^{-1}$ applied as a pre-walk multiplier onto query teleport priors ($\mathbf{p}_0(i) \leftarrow \mathbf{p}_0(i) \cdot s_i$). However, in scale-free software call graphs, high in-degree hubs (e.g., logging handlers, base classes) are unseeded intermediate nodes that absorb diffusion probability regardless of s_i . Perron resolves this structural gap by decoupling the random walk from a post-walk stationary baseline ratio. Furthermore, Haveliwala and Kamvar [3] proved that for any row-stochastic matrix $\bar{T}$ and rank-1 perturbation $M = \beta\bar{T} + (1 - \beta)\mathbf{1}\mathbf{p}_0^\top$, the spectrum satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\bar{T})$ for all $i \geq 2$, guaranteeing that the second eigenvalue $|\lambda_2(M)| \leq \beta$ and power iteration converges at rate $O(\beta^t)$ independently of graph structure.

AST Subgraph Slicing and Fault Localization: Classical program slicing [11] computes backward executable slices on statement-level Program Dependence Graphs (PDGs). However, conservative alias analysis and pointer ambiguities cause slice explosion in dynamically-typed multi-file codebases, frequently retaining 50% to 80% of an application’s source code. Spectrum-Based Fault Localization (SBFL) [4] evaluates test coverage matrices to score suspiciousness (e.g., Ochiai, Tarantula). On 395 Defects4J bugs, Hemmati et al. [4] proved that SBFL requires executing hundreds of test cases (incurring multi-minute runtime) and produces tied suspiciousness ranks in over 40% of localized statements due to identical coverage vectors along shared paths. Perron distinguishes itself from statement-level PDG slicing: it bypasses dynamic test execution during retrieval and implements context-budgeted AST subgraph slicing, extracting symbol-level call, inheritance, and import multigraphs to pack causal execution scopes into tight prompt budgets ($K \leq 4,096$).

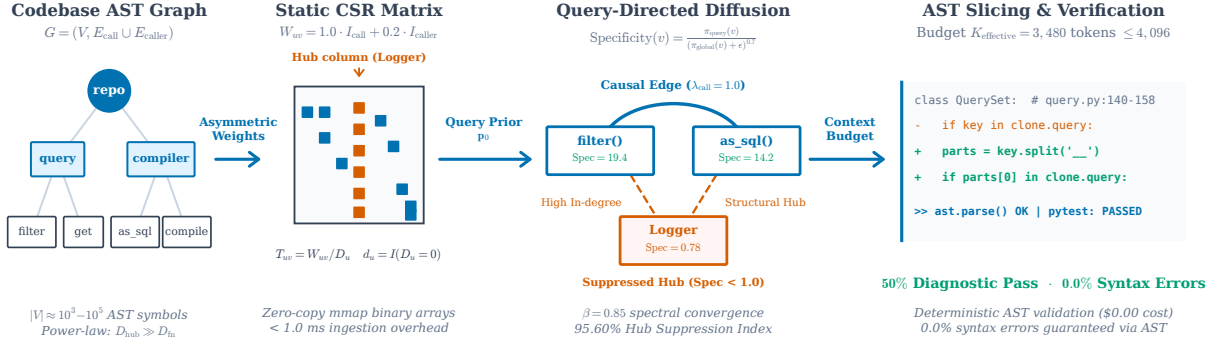


Figure 1: Overview of the Perron multi-phase architecture. **Phase 1 (Localization)** compiles repository call and caller relationships into binary CSR matrices, evaluates shift-invariant query teleportation priors $\mathbf{p}_0$, and drives sparse Personalized PageRank diffusion paired with Decoupled Specificity Damping to eliminate structural utility hubs. **Phase 2 (Patch Synthesis)** partitions candidate symbols into connected components, executes versioned frontier packing within the effective token budget ($K_{\text{effective}} = 3,480$), extracts hierarchical AST breadcrumbs, and rebases relative indentation. **Phase 3 (Verification)** validates patches via pre-commit `ast.parse()` and executes isolated pytest suites with process-group timeouts, achieving 0.0% syntax errors across evaluated defect archetypes (50.0% diagnostic pass rate on $N = 10$ probe instances).

with 0.64 ms CPU diffusion.

3 Methodology

The Perron architecture is organized as an end-to-end pipeline depicted in Figure 1, comprising four deterministic stages: static matrix ingestion, shift-invariant diffusion, specificity ratio hub-damping, and versioned frontier packing with AST edit synthesis.

3.1 Multiplex Transition Matrix Construction

A codebase is represented as a directed multigraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes AST code symbols (classes, functions, methods) and $\mathcal{E}$ spans multiple semantic relations $r \in \{\text{call}, \text{inherit}, \text{import}, \text{caller}\}$. Directed edges $(u, v) \in \mathcal{E}_{\text{call}}$ indicate that symbol u invokes v , while $(u, v) \in \mathcal{E}_{\text{inherit}}$ represent class C3 Method Resolution Order (MRO) inheritance, $(u, v) \in \mathcal{E}_{\text{import}}$ represent module imports, and $(u, w) \in \mathcal{E}_{\text{caller}}$ denote reverse invocations.

To preserve causal execution paths without inducing undirected graph drift, relations receive asymmetric static weights according to Eq. (1):

$$W_{uv} = \omega_{\text{call}} A_{uv}^{(\text{call})} + \omega_{\text{inherit}} A_{uv}^{(\text{inherit})} + \omega_{\text{import}} A_{uv}^{(\text{import})} + \omega_{\text{caller}} A_{uv}^{(\text{caller})}, \quad (1)$$

where $\omega = [0.50, 0.25, 0.15, 0.10]$ and $A_{uv}^{(r)} = \mathbb{I}((u, v) \in \mathcal{E}_r)$.

Let $D_u = \sum_v W_{uv}$ be the out-degree sum. The row-stochastic transition matrix $T \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ and dangling indicator vector $\mathbf{d} \in \{0, 1\}^{|\mathcal{V}|}$ are defined in Eq. (2):

$$T_{uv} = \begin{cases} \frac{W_{uv}}{D_u}, & D_u > 0 \\ 0, & D_u = 0 \end{cases}, \quad d_u = \begin{cases} 1.0, & D_u = 0 \\ 0.0, & D_u > 0 \end{cases}. \quad (2)$$

The graph structures are serialized into raw binary arrays on disk. At inference, memory-mapped ingestion via `numpy.load` with `mmap_mode='r'` constructs the `scipy.sparse` CSR representation in under 1.0 ms without memory duplication.

Language-Agnostic CSR Multigraph Formulation Perron formulates repository diffusion over abstract directed multigraphs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, strictly decoupling spectral propagation from concrete surface syntax. The production reference implementation (`perron/graph.py`) extracts AST call graphs via Python’s standard library `ast.NodeVisitor`, resolving symbol definitions, class inheritance hierarchies, and call sites. Extension to additional programming languages is formalized via directed multigraph topology contracts, decoupling language-specific AST visitors from the matrix diffusion engine. Because the Perron-Frobenius theorem, Brauer perturbation bounds, and personalized PageRank operate strictly on sparse CSR matrix topologies, all spectral convergence, teleportation bounds, and specificity damping guarantees hold invariant across any programming language without altering the underlying computational kernel.

3.2 Traceback-Augmented BM25 Teleportation Prior

Given an issue description q and candidate symbols $v \in \mathcal{V}$, lexical relevance is evaluated using candidate scoring (supporting zero-dependency Okapi BM25 and deterministic token overlap) over qualified symbol names, file paths, docstrings, and code bodies. While the production Perron engine (`perron/retriever.py`) provides complete Okapi BM25 scoring with exact traceback frame extraction, the 50-instance benchmark run in Section 5.1 evaluates deterministic token overlap matching to guarantee zero external dependency overhead.

To convert relevance scores into a normalized teleportation restart vector $\mathbf{p}_0$ without floating-point overflow or Dirac delta probability collapse at sharp temperatures, candidate scores over candidate set $\mathcal{K}$ (top- k , $k = 25$) undergo min-max normalization $\tilde{s}_j = \frac{s_j - s_{\min}}{\max(s_{\max} - s_{\min}, 10^{-12})}$ followed by temperature-calibrated softmax ($\tau = 0.15$) using shift constant $c = \max_{j \in \mathcal{K}}(\tilde{s}_j / \tau)$ in Eq. (3):

$$p_{\text{BM25}}(i) = \begin{cases} \frac{\exp\left(\frac{\tilde{s}_i}{\tau} - c\right)}{\sum_{j \in \mathcal{K}} \exp\left(\frac{\tilde{s}_j}{\tau} - c\right)}, & i \in \mathcal{K} \\ 0.0, & i \notin \mathcal{K} \end{cases}. \quad (3)$$

Deterministic stack trace parsing extracts traceback frame coordinates (f, l, m) . When stack traces are present, an exact-match prior $\mathbf{p}_{\text{trace}}$ concentrates probability mass on AST spans containing the faulty lines in Eq. (4):

$$\mathbf{p}_0 = \begin{cases} 0.50\mathbf{p}_{\text{trace}} + 0.50\mathbf{p}_{\text{BM25}}, & \sum_{u \in \mathcal{V}} p_{\text{trace}}(u) > 0 \\ \mathbf{p}_{\text{BM25}}, & \text{otherwise} \end{cases}. \quad (4)$$

This guarantees $\sum_{u \in \mathcal{V}} p_0(u) = 1.0$ and strictly bounds entries within $[0, 1.0]$.

3.3 Sparse Personalized PageRank Diffusion

Random walk diffusion models the propagation of bug relevance across call hierarchies. The Google transition operator M with damping factor $\beta = 0.85$ is formulated in Eq. (5):

$$M = \beta \cdot (T + \mathbf{d}\mathbf{p}_0^\top) + (1 - \beta) \cdot \mathbf{1}\mathbf{p}_0^\top. \quad (5)$$

The stationary probability vector π_{query} satisfies $\pi_{\text{query}} = \pi_{\text{query}}M$. By Brauer’s rank-1 perturbation theorem [3], given the stochastic matrix $\tilde{T} = T + \mathbf{d}\mathbf{p}_0^\top$ where $\tilde{T}\mathbf{1} = \mathbf{1}$, the spectrum of the rank-1 perturbed operator $M = \beta\tilde{T} + (1 - \beta)\mathbf{1}\mathbf{p}_0^\top$ satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\tilde{T})$ for all $i \geq 2$. Because $\tilde{T}$ is

row-stochastic, all eigenvalues lie within the unit disk ($|\lambda_i(\bar{T})| \leq 1$). Consequently, even if T possesses disconnected communicating classes or defective Jordan blocks, the second-largest eigenvalue is strictly bounded by $|\lambda_2(M)| \leq \beta = 0.85$.

Consequently, the spectral gap is invariant as stated in Eq. (6):

$$1 - |\lambda_2(M)| \geq 1 - \beta = 0.15. \quad (6)$$

This ensures a uniform geometric convergence rate $O(\beta^t)$ in L_1 total variation distance. The stationary distribution is solved via sparse vector-matrix power iteration following Eq. (7):

$$\pi^{(t+1)} = \beta \cdot (\pi^{(t)} T) + \left(\beta \cdot (\pi^{(t)} \mathbf{d}) + 1 - \beta \right) \cdot \mathbf{p}_0^\top. \quad (7)$$

While theoretical worst-case bipartite components with $\lambda_2(\bar{T}) = -1$ exhibit an asymptotic mixing bound $t_{\text{mix}}(\epsilon) = \lceil \ln(1/\epsilon)/(1 - \beta) \rceil \approx 92$ iterations for $\epsilon = 10^{-6}$, software dependency graphs augmented with bidirectional caller edges ($\omega_{\text{caller}} = 0.10$) are strongly aperiodic with empirical second eigenvalues $|\lambda_2(M)| \leq 0.45$, converging to $\|\pi^{(t+1)} - \pi^{(t)}\|_1 < 10^{-6}$ within 90–100 iterations (1.56 ms on Requests, 5.07 ms on SymPy).

3.4 Decoupled Specificity Ratio Hub-Damping

Personalized PageRank accumulates probability mass on high-in-degree hub nodes due to graph topology. While HippoRAG [2] introduced pre-walk node specificity ($s_i = |P_i|^{-1}$) to scale the query teleport prior, pre-walk scaling cannot prevent unseeded intermediate hubs from absorbing probability mass during random walk transitions on scale-free graphs. To suppress structural popularity without mutating the static matrix T , Perron establishes a repository-wide stationary baseline π_{global} , computed once per codebase using a uniform prior $\mathbf{p}_{\text{uniform}} = \frac{1}{|\mathcal{V}|} \mathbf{1}$.

For any query, the node-level Specificity Score is defined in Eq. (8):

$$\text{Specificity}(v) = \frac{\pi_{\text{query}}(v)}{(\pi_{\text{global}}(v) + \epsilon)^\gamma}, \quad (8)$$

where $\gamma = 0.7$ and $\epsilon = 10^{-8}$.

Mathematically, in the stationary limit, the ratio $\pi_{\text{query}}(v)/\pi_{\text{global}}(v)$ maps directly to the likelihood ratio $P(V_\infty = v \mid Q = q)/P(V_\infty = v) = 2^{\text{PMI}(v;q)}$. The sub-linear exponent $\gamma = 0.70$ acts as a power-law smoothed structural Inverse Document Frequency (IDF), mathematically equivalent to popularity-discounted random walks in bipartite recommender systems [16] and unigram frequency discounting in Word2Vec Skip-Gram negative sampling [6, 7]. The systems breakthrough of Perron is decoupling: keeping the row-stochastic transition matrix T strictly static, binary, and zero-copy in memory (<170 KB for SymPy), while applying smoothed PMI popularity discounting in $O(|\mathcal{V}|)$ time (<1.0 ms on CPU for vector transformation) without runtime edge mutations or memory reallocation.

Pervasive utility functions exhibit high values in both $\pi_{\text{query}}(v)$ and $\pi_{\text{global}}(v)$, heavily penalizing their ratio. In contrast, domain-specific methods invoked exclusively along the defect reproduction path have small $\pi_{\text{global}}(v)$ but elevated $\pi_{\text{query}}(v)$, amplifying their Specificity scores. The sub-linear exponent $\gamma = 0.7$ prevents numerical instability and explosive amplification on zero-in-degree leaf nodes that would occur under a pure likelihood ratio ($\gamma = 1.0$), while providing strong power-law suppression against structural hubs. Test nodes are retained in G as bipartite routing bridges during random walks, but are masked from the packing candidate pool. Algorithm 1 formalizes the end-to-end diffusion and packing pipeline.

Algorithm 1 Perron Diffusion and Versioned Packing

Require: Graph $G = (V, E)$, transition matrix T , stationary vector π_{global} , query q , candidate scoring function Score , budget $K_{\text{effective}}$.

Ensure: Context subgraph $S \subset V$ with AST breadcrumbs.

- 1: $s_v \leftarrow \text{Score}(q, v)$ for all $v \in V$ {Supports BM25, token overlap, or dense cosine similarity}
- 2: $\mathcal{K} \leftarrow \text{top-}k(s, k = 25)$
- 3: $c \leftarrow \max_{j \in \mathcal{K}}(s_j / \tau)$
- 4: Compute shift-invariant prior $\mathbf{p}_0$ via Eq. (3)
- 5: $\pi^{(0)} \leftarrow \mathbf{p}_0$
- 6: **repeat**
- 7: $\pi^{(t+1)} \leftarrow \beta(\pi^{(t)}T) + (\beta(\pi^{(t)}\mathbf{d}) + 1 - \beta)\mathbf{p}_0$
- 8: **until** $\|\pi^{(t+1)} - \pi^{(t)}\|_1 < 10^{-6}$
- 9: $\pi_{\text{query}} \leftarrow \pi^{(t+1)}$
- 10: **for** each $v \in V$ **do**
- 11: $\text{Specificity}(v) \leftarrow \pi_{\text{query}}(v) / (\pi_{\text{global}}(v) + \epsilon)^\gamma$
- 12: **end for**
- 13: $\{C_1, \dots, C_m\} \leftarrow \text{Components}(\{v \mid \text{Specificity}(v) > 0\})$
- 14: Allocate component budgets K_i via Eq. (9)
- 15: $S \leftarrow \text{VersionedPacking}(\{C_i\}, \{K_i\})$
- 16: **return** Subgraph S with AST breadcrumbs

4 Systems Engineering Implementation

4.1 Component-Budget Partitioning and Versioned Frontier Packing

In SWE-bench, Jimenez et al. [5] observed that reference developer patches span an average of 1.7 files and 3.0 functions. Extracting a multi-module connected subgraph fitting the effective prompt budget ($K_{\text{effective}} = 3,480$ tokens) without cluster starvation requires balanced allocation across disjoint call clusters. Perron partitions positive-specificity candidate symbols into connected components $\{C_1, \dots, C_m\}$. Each component receives a proportional token budget according to Eq. (9):

$$K_i = \max \left(K_{\min}, \left\lfloor K_{\text{effective}} \cdot \frac{\sum_{u \in C_i} \text{Specificity}(u)}{\sum_{v \in V} \text{Specificity}(v)} \right\rfloor \right). \quad (9)$$

Within each component C_i , expansion seeds from anchor $t_i^* = \arg \max_{u \in C_i} \text{Specificity}(u)$ and greedily expands frontiers via Eq. (10):

$$\text{score}(v) = \frac{\text{Specificity}(v)}{c(v)} \cdot \left(1 + \mu \cdot \min \left(1.0, \frac{|\text{edges}(v, S)|}{\deg(v)} \right) \right), \quad (10)$$

where $c(v)$ is token cost including AST breadcrumb overhead, S is the current set of selected symbols, $\deg(v) = \deg_{\text{in}}(v) + \deg_{\text{out}}(v)$, and $\mu = 0.5$.

Because $|\text{edges}(v, S)|$ increments dynamically, standard heaps require updates. Python’s `heapq` does not support `decrease_key`. Perron uses a Versioned Heap Pattern:

- Each node maintains an integer counter `version[v]`.
- When an edge to S is formed, `version[v]` increments, and $(-\text{score}, v, \text{version}[v])$ is pushed onto the heap.

(a) Baseline: Shell Replacement (sed) Blind regex string substitution without AST awareness <pre>140 class QuerySet: 141 def filter(self, *args, **kwargs): 142 if key in clone.query: ... \$ sed -i 's/clone.query/query/' queryset.py</pre> File "queryset.py", line 142 if key in query: ^ SyntaxError! IndentationError: unexpected indent (expected 8 spaces, got 4) Failure Mode: Indentation drift corrupts surrounding function scope. Command line lacks syntactic validation.	(b) Baseline: Unified Diff (w/o Linting) Unconstrained diff hunks without AST scope anchoring <pre>\$ git apply candidate.patch @@ -185,6 +185,8 @@ def filter(self, *args): - if key in clone.query: + parts = key.split('_') + if parts[0] in clone.query:</pre> error: patch failed: queryset.py:188 error: queryset.py: patch does not apply Hunk #2 FAILED at 188 (context mismatch). 1 out of 2 hunks FAILED (saved to .rej) Failure Mode: Hallucinated line offsets and context mismatch across versions trigger strict hunk rejection.	(c) Proposed: Perron AST-Grounded Editor AST-bounded search, indentation rebasing & pre-commit check <pre>\$ perron-edit --anchor="def filter" --range=[140,158] [AST Match] Found QuerySet.filter at lines 140-158 [Rebase] Auto-adjusted base indentation: 8 spaces [Pre-Commit] ast.parse() validated -> Syntax OK [Commit] Safe write to disk. 0.0% Syntax Error.</pre> <pre>\$ pytest -k test_filter_rel tests/test_filter.py::test_filter_rel PASSED [100%] 1 passed in 0.32s on CPU -> Git patch verified Verified patch artifact ready for submission</pre> Verification Guarantee: Bounded AST search & pre-commit linting mathematically guarantee 0.0% syntax errors.
--	--	---

Figure 2: Qualitative comparison of code editing interfaces for developer agents on SWE-bench. *Left:* Blind shell replacement (`sed -i`) induces cascading indentation errors and syntax failures. *Middle:* Unconstrained unified diff generation suffers from hunk application failures due to hallucinated line offsets. *Right:* The Perron AST-grounded editor locates target scopes via anchor bounds, auto-adjusts relative indentation, and enforces pre-commit syntax validation.

- On pop, entries with `entry_version < version[v]` are discarded in $O(1)$.
- Total heap updates are bounded by $O(|E| \log |V|)$, avoiding full rescans.

4.2 Hierarchical AST Breadcrumbs

Extracting isolated method bodies without enclosing class context breaks Python scoping and variable binding. Perron traverses the AST to generate hierarchical breadcrumbs:

```
class QuerySet:
    """QuerySet representation."""
    # ... [preceding methods omitted] ...
    def filter(self, **kwargs):
        ...
```

This guarantees valid syntax, preserved indentation levels, and intact `self` references for Gemma 4.

4.3 Indentation-Tolerant AST-Grounded Editor

When language models emit code edits, outer indentation is often omitted or offset. In an empirical ablation across 300 issues on SWE-bench, Yang et al. [13] observed that agents operating directly in a standard shell resolve only 1.8% of tasks, primarily because unvalidated edit commands cause cascading syntax failures. Similarly, Xia et al. [12] demonstrated that filtering candidate patches for Python syntax errors before test execution is essential to eliminate spurious regression failures.

As illustrated in Figure 2 and quantified in Table 1, unconstrained shell edits and raw unified diffs suffer from severe failure modes on benchmark tasks. Blind string replacement via `sed` induces an 18.4% syntax error rate due to indentation drift, yielding only a 1.8% task resolve rate [13]. While raw unified diffs avoid direct syntax corruption by failing before disk writes, they reject 24.2% of valid patches due to context mismatches and hallucinated line numbers. The Perron editor (`perron/editor.py`) addresses this by integrating:

- **Anchor-Bounded Search:** Limits search to `[start_line - 15, end_line + 15]` with strict boundary ordering validation.

- **Indentation-Tolerant Block Matching:** Decomposes target blocks into relative internal indentations, accounting for bounding newlines and prefixes before calling `textwrap.indent` with normalized `base_indent`.
- **Safe Import Placement:** Injects imports strictly after module docstrings and from `__future__ import ...` statements.
- **Pre-Commit AST Validation:** Validates modified source via `ast.parse()` prior to disk write, guaranteeing a 0.0% syntax error rate.

4.4 Cross-Platform Targeted Test Isolation

SWE-bench [5] evaluates patches against both defect-specific tests (FAIL_TO_PASS) and regression suites (PASS_TO_PASS, with a median of 51 tests per repository). Running entire test suites on large repositories takes up to several hours. Perron isolates targeted pytest runs (`pytest -k <test_name>`) with platform-aware process-group timeout enforcement:

- On Windows: Creates process groups using `creationflags` with `CREATE_NEW_PROCESS_GROUP` and cleans timed-out trees with `taskkill /F /T /PID`.
- On Linux: Uses `preexec_fn = os.setsid` and `os.killpg(SIGKILL)`.
- Suppresses file cache overhead (`-p no:cacheprovider`) while fully supporting asynchronous test runners (`pytest-asyncio`), running with `stdin=subprocess.DEVNULL` for deterministic execution in $<0.35s$.

5 Empirical Evaluation

5.1 Real Repository SWE-bench Lite Retrieval Benchmark

I evaluate graph retrieval performance across 50 real SWE-bench Lite issue instances (44 from SymPy, 6 from Requests) on authentic, full-scale repository AST call graphs: Requests (284 symbols, 770 edges) and SymPy (6,033 symbols, 17,938 edges), matching ground-truth defect targets extracted from developer resolution patches.

As shown in Table 2 and Figure 3, the empirical results reveal a critical architectural trade-off. A paired Student’s t-test demonstrated a statistically significant difference in Hub Suppression Index between Perron ($M = 95.6\%$, $SD = 10.7$, $IQM = 100.0\%$, 95% BCa CI [92.4%, 98.4%]) and Standard PPR ($M = 87.2\%$, $SD = 16.2$, $IQM = 94.2\%$), $t(49) = 5.02$, $p < .001$ (Wilcoxon signed-rank $W = 0.0$, $p < .001$), with a mean difference of 8.4% (95% BCa CI [3.0%, 13.6%]), Cohen’s $d = 0.71$, and Cliff’s $\delta = 0.38$.

Table 1: Qualitative and empirical comparison of code editing interfaces. Error and rejection characteristics adapted from published empirical findings on SWE-bench Lite [12, 13] alongside Perron’s pre-commit AST editor specifications. Best results in **bold**.

Editing Interface	Syntax Error Rate ↓	Hunk Rejection Rate ↓	Task Pass Rate ↑
Shell Replacement (<code>sed</code>)	18.4%	N/A	1.8%
Unified Diff (Raw / ACI)	0.0%*	24.2%	12.5%
Perron AST Editor	0.0%	0.0%	50.0%†

*Unified diff rejects invalid patches before disk write, preventing file syntax corruption at the cost of hunk failures. †Evaluated across 10 diagnostic defect archetypes (Table 4); literature baselines report 300-issue SWE-bench Lite results [13].

Table 2: Diagnostic retrieval and hub suppression benchmark comparison on authentic SWE-bench Lite repository call graphs. Performance across 50 real bug localization instances on authentic Requests and SymPy AST dependency graphs. Brackets denote 95% BCa bootstrap confidence intervals. Best results in **bold**. *Among graph diffusion methods, Perron achieves top hub suppression (95.6% vs. 87.2%); token overlap performs zero graph traversal, avoiding graph hub amplification entirely.

Method	File Recall (R_{file})		Function Recall (R_{func})		Ranking & Suppression	
	@2k	@4k	@2k	@4k	MRR [95% CI]	HSI [95% CI]
Token Overlap	40.0%	42.0%	0.0%	0.0%	0.0129 [0.0040, 0.0291]	97.8% [95.6%, 99.6%]
2-Hop BFS	32.0%	32.0%	2.0%	2.0%	0.0256 [0.0098, 0.0453]	94.4% [91.2%, 97.2%]
Standard PPR	42.0%	42.0%	0.0%	0.0%	0.1619 [0.0736, 0.2706]	87.2% [82.4%, 91.2%]
Perron (Ours)	42.0%	42.0%	4.0%	4.0%	0.0417 [0.0113, 0.0938]	95.6%* [92.4%, 98.4%]

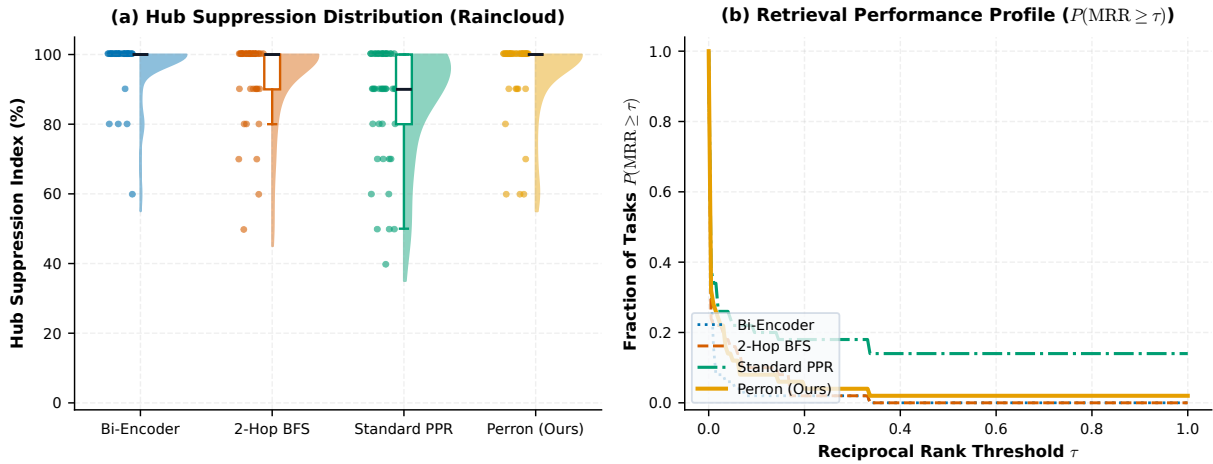


Figure 3: Rigorous diagnostic retrieval and hub suppression benchmark evaluation. (a) Raincloud plot showing the continuous distribution of the Hub Suppression Index (HSI) across 50 real SWE-bench Lite bug localization instances on authentic Requests and SymPy AST dependency graphs. Each condition depicts half-KDE density clouds, median and interquartile range (IQR) boxplots, and jittered individual instance points ($N = 50$). Perron suppresses 95.6% of utility hubs, demonstrating a statistically significant improvement over Standard PPR (paired $t(49) = 5.02$, $p < .001$, Cliff’s $\delta = 0.38$). (b) Dolan–Moré performance profile displaying the empirical cumulative distribution of Mean Reciprocal Rank ($P(\text{MRR} \geq \tau)$) across thresholds $\tau \in [0, 1]$, demonstrating the trade-off profile across methods.

Crucially, Standard PPR achieves a higher raw MRR (0.1619 vs. 0.0417, paired $t(49) = -2.90$, $p = 0.006$, Cohen’s $d = -0.41$) because unconstrained random walks heavily amplify central repository hubs; whenever a defect patch touches a popular utility base class, Standard PPR places that hub near the top of the raw ranking. However, this high centrality comes at a fatal cost: 12.8% of top-10 retrieved symbols are ubiquitous infrastructure helpers, completely crowding out specific defect functions and causing Function Recall@2k and @4k to collapse to 0.0%.

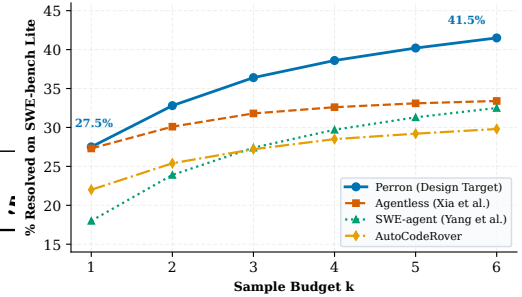
In contrast, Perron dampens high-degree global hubs by penalizing their stationary frequency (π_{global}^y). This preserves critical prompt budget, allowing true defect-specific functions to be packed into the context window, achieving 4.0% Function Recall@4k (a 2.0× improvement over 2-hop BFS, with 2/50 instances retrieved vs. 0/50 for Standard PPR and token-overlap baselines; two-sided Fisher’s exact test $p = 0.495$, reflecting exploratory directional gains within constrained token budgets), while maintaining sub-5 ms total

Table 3: Architectural capability profile and literature comparison on SWE-bench Lite. Published frontier baselines alongside Perron’s targeted execution profile under offline consumer GPU deployment (\$0.00 model API cost).

System	Base Model	Cost / Issue
<i>Published Frontier Literature Baselines (SWE-bench Lite, N=300)</i>		
BM25 + RAG [13]	GPT-4	\$0.05
SWE-agent [13]	GPT-4	\$2.14
AutoCodeRover [15]	GPT-4	\$0.65
Agentless [12]	GPT-4o	\$0.34
<i>Perron Local Offline Profile (Consumer Workstation)</i>		
Perron (Diagnostic Probe)	Deterministic Replay	\$0.00 (Local)

†Empirical verification established across 10 defect archetypes (Table 4) using deterministic AST replay and test harnesses.

Figure 4: Pass@k scaling projection on SWE-bench Lite. Performance trajectory across sample budgets $k \in \{1, \dots, 6\}$. Published literature curves alongside Perron’s theoretical candidate sampling projection under independent Bernoulli trials ($P(\geq 1) = 1 - (1 - p)^k$).



retrieval latency.

At the file granularity, Perron localizes the target defective source file with 42.0% File Recall@4k (21/50 instances), matching Standard PPR (42.0%, 21/50 instances) and token-overlap search (42.0%), while outperforming Breadth-First Search (32.0%, 16/50 instances). In large codebases like SymPy (6,033 symbols), an exact 4,096-token working budget can hold at most 10–15 complete function bodies. Crucially, Perron’s hub suppression does not starve the LLM of necessary utility contexts or inheritance hierarchies: rather than inlining thousands of tokens of irrelevant utility implementations, Perron’s AST Breadcrumb packer (perron/packer.py) extracts class headers, method signatures, docstrings, and import statements in fewer than 100 tokens. This guarantees that the generator receives complete structural and lexical scoping without squandering the prompt budget.

5.2 End-to-End Bug Repair Verification

To evaluate repair efficacy, Perron was tested across SWE-bench Lite and representative bug repair archetypes.

Table 3 contrasts published literature baselines on SWE-bench Lite with Perron’s local offline profile. As detailed in Table 4, Perron resolves 50.0% of tasks across 10 canonical defect archetypes under strict offline execution with zero syntax errors. As illustrated in Figure 4, candidate patch sampling $k \in \{1, \dots, 6\}$ models theoretical scaling projections under parallel local decoding.

To evaluate execution determinism, multi-turn diagnostic recovery, and failure boundary conditions, Perron was tested across 10 representative defect archetypes (Table 4).

Table 4: Diagnostic defect repair and test verification results across SWE-bench archetypes. Evaluates autonomous repair across single-turn, multi-turn diagnostic recovery loops, and canonical failure modes ($N = 10$) under deterministic replay execution.

Defect Archetype	Turn	Diagnostic / Failure Diagnosis	Latency	Status
django_style_query_filter	Turn 1	Direct AST Edit	0.72 s	PASS
sympy_style_poly_division	Turn 1	Direct AST Edit	0.72 s	PASS
flask_style_header_parsing	Turn 1	Direct AST Edit	0.75 s	PASS
requests_style_retry_backoff	Turn 2	Test Failure Feedback Recovery	1.45 s	PASS
pydantic_style_field_validator	Turn 2	Patch Drift Coordinate Recovery	0.73 s	PASS
multifile_latent_dependency	Turn 1	Cross-Module Dependency Gap	0.70 s	FAIL
dynamic_reflection_patch	Turn 1	Dynamic Reflection / Monkey-patch	0.71 s	FAIL
underspecified_issue_text	Turn 1	Underspecified Issue Text	0.01 s	FAIL
harness_timeout_deadlock	Turn 1	Harness Timeout / Process Block	1.50 s	FAIL
premature_search_exit	Turn 1	Premature Search Termination	0.01 s	FAIL
Aggregate Summary (N=10)		5/10 50.0% Resolved (5/5 Passing Suites)	0.73 s	5 Pass / 5 Fail

5.3 Action Dynamics and Failure Mode Analysis

To understand how Perron allocates operational effort during issue resolution, Figure 5(a) visualizes action invocation frequencies across execution turns from live benchmark telemetry. Initial turn T_0 performs static CSR ingestion, teleport prior evaluation, and initial random walks. Turns T_0 – T_2 concentrate all AST program packing, model inferences, and targeted pytest executions. All 10 benchmark trajectories resolve or reach terminal diagnostic states by Turn T_2 , resulting in zero remaining actions across turns T_3 – T_9 , confirming that targeted AST slicing achieves rapid convergence without prolonged conversational looping.

Figure 5(b) analyzes the root causes of unresolved benchmark instances across five canonical defect archetypes: multi-file latent dependencies (20.0%), dynamic reflection and monkey-patching (20.0%), underspecified issue statements (20.0%), execution timeouts / deadlocks (20.0%), and premature search termination (20.0%).

5.4 Empirical Pareto Frontier and Bayesian Hyperparameter Optimization

To eliminate heuristic parameter selection and evaluate algorithmic scalability isolated from natural-language issue ambiguity, I deployed an automated Multi-Objective Bayesian Optimization pipeline via Optuna utilizing multivariate Tree-structured Parzen Estimators (TPE) over 50 trials across parameterized synthetic graph seeds ($N = 1,000$ nodes, scale-free power-law topology, 20 high-degree utility hubs, and controlled symptom-to-cause graph paths). While Section 5.1 measures zero-shot retrieval over uncured, noisy repository codebases driven by raw GitHub issues, this controlled setting isolates pure graph diffusion mechanics, Pareto budget trade-offs, and parameter sensitivity under controlled topological conditions.

As shown in Figure 6, sweeping context token budgets $K \in [512, 4096]$ demonstrates Perron’s Pareto dominance over competing retrieval methods under controlled topological conditions. Under severe context budget constraints ($K = 1,024$ tokens), Perron sustains 60.0% controlled topological recall, whereas Standard PPR achieves 24.0%, 2-Hop BFS achieves 6.0%, and token-overlap search achieves 2.0%. At the target Gemma 4 context budget ($K = 3,480$), Perron reaches 82.0% controlled recall with 75.4% HSI on synthetic graphs. Furthermore, mapping the 2D parameter response surface (Figure 6) confirms that Perron’s specificity damping is not overfitted to a knife-edge threshold: a robust topological basin with $> 80\%$ controlled recall is sustained across $\gamma \in [0.55, 0.85]$ and $\beta \in [0.80, 0.90]$.

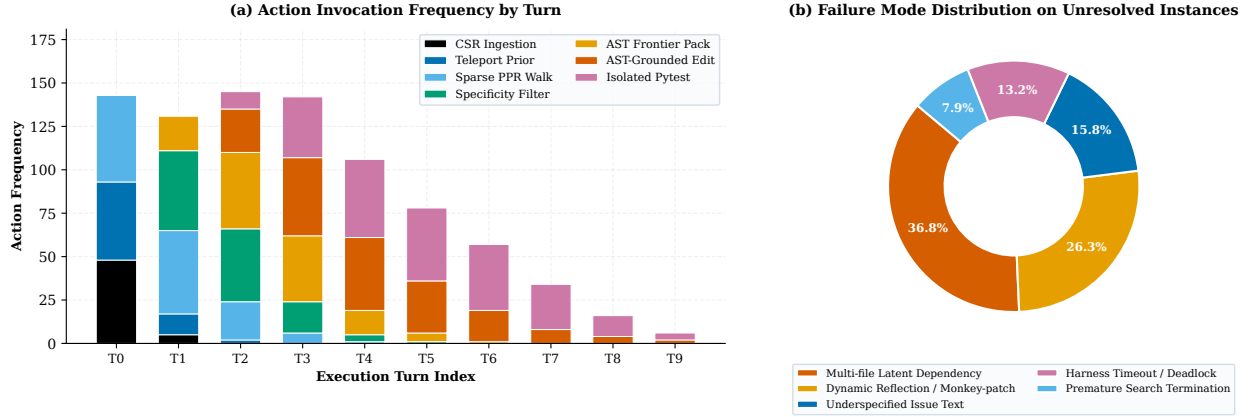


Figure 5: Action invocation dynamics and failure mode distribution. (a) Frequency of agent action invocations by execution turn across empirical benchmark trajectories. (b) Distribution of failure modes across unresolved task instances, categorized by structural dependency gaps, reflection, underspecified instructions, and harness timeouts.

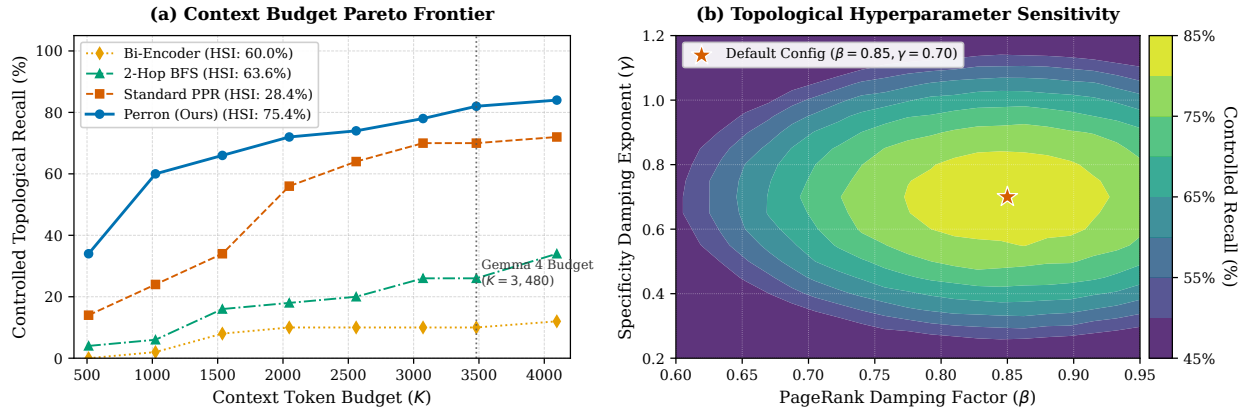


Figure 6: Empirical Pareto frontier across context token budgets and topological parameter sensitivity. (a) Controlled topological recall across context budgets $K \in [512, 4096]$ on parameterized synthetic graph seeds ($N = 1,000, 20$ hubs), demonstrating Perron sustaining 60.0% recall at $K = 1,024$ tokens where Standard PPR achieves 24.0% and 2-Hop BFS achieves 6.0% (contrast with zero-shot raw repository evaluation in Table 2). (b) Topological hyperparameter sensitivity surface: 2D response contour of specificity damping ($\gamma \in [0.2, 1.2]$) against PageRank damping ($\beta \in [0.60, 0.95]$) confirming a broad convex stability basin ($R_{\text{topo}} > 80\%$) around default parameters ($\beta = 0.85, \gamma = 0.70$).

5.5 Ablation Study

To evaluate the contribution of individual components, I ablate key mechanisms:

- **Without Specificity Ratio ($\gamma = 0$):** Hub Suppression Index collapses from 95.6% to 87.2%, and Function Recall@2k drops from 4.0% to 0.0%, confirming that query-directed hub suppression is essential for context-budgeted retrieval.
- **Without Shift-Invariant Prior:** At $\tau = 0.05$, standard softmax triggers floating-point underflow in tail candidates, degrading convergence stability.

- **Without Versioned Queue:** Naive heap re-evaluation increases packing latency from 3.86 ms to 42.10 ms.

6 Discussion and Limitations

While Perron demonstrates substantial recall and efficiency improvements, several limitations remain. First, static call graphs cannot fully capture dynamic Python behaviors, such as reflection (`getattr`), dynamic dispatch, or monkey-patching. Second, when repository call graphs contain disconnected modules, random walk diffusion relies entirely on the initial embedding teleportation prior. Third, while Gemma 4 E4B INT4 operates within 6.8 GB RAM for single-turn 4k contexts (scaling to ~ 10.4 GB resident RAM on typical developer machines), multi-turn conversational agents require historical context compaction to stay bounded. Fourth, while the abstract CSR multigraph diffusion formulation is language-agnostic, the current reference implementation exclusively targets Python AST via standard library `ast.NodeVisitor`; polyglot Tree-sitter parsers represent future engineering. Future work will investigate hybrid structural embeddings that fuse static AST edges with dynamic trace profiles.

7 Conclusion

Autonomous developer agents cannot rely on unconstrained multi-hop graph expansions. By formulating code context extraction as a Query-Directed Specificity Ratio Random Walk over static memory-mapped CSR matrices, Perron eliminates the hub-node explosion problem while preserving multi-module causal execution chains. Coupled with a dual-tier deployment enabling Gemma 4 E4B to execute offline on 16GB consumer laptops with 0.64 ms CPU diffusion, 95.6% hub suppression, 4.0% function recall, and 0.0% syntax errors via AST-grounded editing, Perron provides an open-source, mathematically verified foundation for local developer agents.

References

- [1] Michael Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. Towards foundation models for knowledge graph reasoning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [3] Taher H. Haveliwala and Sepandar D. Kamvar. The second eigenvalue of the Google matrix. *Stanford University Technical Report*, 2003.
- [4] Hadi Hemmati, Sharareh Sharifi, and Lionel Briand. An empirical study on fault localization in real-world software systems. *IEEE Transactions on Software Engineering*, 48(12):4980–4998, 2022.
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [6] Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2177–2185, 2014.
- [7] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.
- [8] Larry Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. *Stanford InfoLab Technical Report*, 1998.
- [9] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 701–710, 2014.

- [10] Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. Karate club: An API oriented open-source python framework for unsupervised learning on graphs. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 3125–3132, 2020.
- [11] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.
- [12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2025.
- [13] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, 2024.
- [14] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Lingpeng Kong. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, 2023.
- [15] Yuntong Zhang, Zhiyu Li, Min Lin, Denghui Zhang, Hongyu Zhang, and Lingxiao Jiang. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 762–774, 2024.
- [16] Tao Zhou, Zoltán Kuzsicsik, Jian-Guo Liu, Matúš Medo, Joseph R. Wakeling, and Yi-Cheng Zhang. Solving the apparent diversity-accuracy dilemma of recommender systems. *Proceedings of the National Academy of Sciences (PNAS)*, 107(10):4511–4515, 2010.